

SnapRoute

Technical Project Report

Category: Windows Desktop Utility (Screen Capture, Optical Character Recognition, and Visual Search Routing)

Language/Framework: Python 3 + PyQt6

Distribution Model: Standalone, Portable Windows Executable (`.exe`)

1. Executive Summary

SnapRoute is a premium, ultra-lightweight, 100% portable screen cropping and text recognition (OCR) utility. Triggered by a global hotkey, it freezes the display with a cinematic fade-in, allows the user to crop any region with a glowing neon border, and provides an instant HUD (Heads-Up Display) overlay. The HUD smart-routes the selection: copying text or images to the clipboard, launching Google searches for text, or uploading graphics to a secure, ephemeral host to trigger a Google Lens reverse visual search.

SnapRoute is designed with an emphasis on **zero-dependency portability** (fully bundling the Tesseract-OCR binary engine) and **premium UX aesthetics** (fluid hardware-accelerated opacity animations, zero layout-shifting jitter).

2. Key Product Features

Feature	Description	Engineering Implementation
Cinematic Dim Fade-in	On trigger, the display freezes and dim-fades into a dark navy translucent overlay.	Multi-threaded frame grabber with a <code>QTimer</code> - driven alpha increment loop (80 FPS).
High-Visibility Cursor	Custom neon-emerald crosshair target with dark drop-shadows for visibility on any background.	Pre-baked 36x36 transparent <code>QPixmap</code> drawn with dual-color <code>QPainter</code> layout.
Dynamic Glassmorphic HUD	Translucent HUD dock overlay displaying context-aware action buttons.	Custom frameless <code>QWidget</code> with translucent backgrounds and HSL glassmorphism styles.

Feature	Description	Engineering Implementation
Context-Aware Smart Routing	Automatically detects readable characters and adjusts HUD actions dynamically.	OCR characters are evaluated. If ≥ 2 characters are found, "Search Text" is enabled and Copy label changes.
Uguu-First Upload	Sequentially attempts file uploads to unblocked direct image hosts to trigger Google Lens.	Asynchronous <code>QThread</code> utilizing Uguu.se, Catbox.moe, and Pixeldrain.com.
Zero-Configuration Portability	Single executable delivery with no Python runtime or external installations required.	PyInstaller bundling with portable <code>Tesseract-OCR</code> asset folder mappings.

3. System Architecture & Workflows

SnapRoute is architected to keep the main GUI thread completely fluid and responsive (0ms delay) by offloading blocking operations (OCR, Network Uploads, Keyboard Hook monitoring) to background threads.

Execution Flow

```
[Global Hotkey: Ctrl+Shift+S]
└─> (Thread-safe queue callback) Ignite Sequence
    └─> Screen Capture & Overlay Launch
        ├──> (QTimer 12ms) Cinematic Background Dim Fade-in
        └─> (Mouse Press & Drag) Draw Glowing Neon Selection Box
            └─> (Mouse Release) Mount HUD Overlay & Save Crop
                ├──> (Spawn Thread 1) OCRWorker text extraction
                │   └─> (Finished) Enable Text Search / Copy Label
                └─> (Instant Display) HUD Action Menu
                    ├──> [Click: Copy] → Clipboard Action
                    ├──> [Click: Search Text] → Google Search
                    └─> [Click: Search Image] → (Thread 2) UploadWorker
                        └─> (Finished) Google Lens uploadbyurl
```

3.1 Core Components

1. **System Tray Daemon (`main.py`):** Starts the application event loop, creates the taskbar tray menu, triggers the startup splash animation, and spawns the global hotkey listener.
2. **Keyboard Polling Manager:** Monitors a thread-safe `queue.Queue` from a background thread to intercept `Ctrl + Shift + S` triggers without blocking the main Qt thread.
3. **Screen Overlay UI (`box_search.py`):** Creates the frameless fullscreen window, captures the primary display screenshot, runs drawing/painting cycles, and controls HUD positioning.

4. **OCR Worker Thread (`OCRWorker`)**: Runs `pytesseract` image-to-string operations on the cropped PNG in the background.
5. **Upload Worker Thread (`UploadWorker`)**: Uploads the image to anonymous hosts sequentially using standard Python libraries to prevent main window freeze.

4. Key Engineering Challenges & Solutions

4.1 Windows High-DPI Display Scaling Mismatch

- **The Problem**: On high-resolution displays (4K/2K) with Windows scaling active (e.g., 125% or 150%), coordinates returned by mouse events are in **logical pixels**, while screenshot coordinates are captured in **physical pixels**. Manually multiplying logical coordinates by display scaling factors caused a double-scaling bug, yielding blank/offset crops.
- **The Solution**: We removed manual DPI multipliers and mapped selection bounds directly. PyQt6's `QPixmap.copy` automatically handles logical-to-physical coordinate mappings natively. By passing coordinates cleanly without scaling, crops became 100% pixel-perfect.

4.2 ISP/DNS-Level Blocking of Image Hosts

- **The Problem**: Direct visual search with Google Lens requires uploading the cropped region to a public URL. Popular image hosts (like Catbox or Pixeldrain) are blocked at the DNS/ISP level by major providers in certain countries (like Jio and Airtel in India), causing uploads to time out.
- **The Solution**: We designed a multi-host, sequential upload worker. It uses **Uguu.se** (an open-source, keyless developer host that is unblocked globally and deletes files after 24 hours) as the primary target. If Uguu fails, the thread automatically falls back to Catbox.moe, and then Pixeldrain.com, ensuring global search reliability.

4.3 Jitter and Reflow of interactive HUD Buttons

- **The Problem**: When OCR finished asynchronously in the background, the HUD dynamically resized and re-centered itself to accommodate the new "Search Text" button. This caused the HUD window to jump on Windows, and shifted buttons right out from under the user's cursor as they moved their mouse to click.
- **The Solution**: The HUD was redesigned with a static, fixed width (`440px`). All buttons remain in fixed coordinates. While OCR runs, the "Search Text" button is placed in an elegant, low-opacity **disabled state**. Once OCR completes, the button is simply enabled, creating a lag-free, non-shifting state transition.

4.4 Stays-on-Top Window Hierarchy Conflict

- **The Problem**: Triggering the hotkey immediately after launching caused the stays-on-top Welcome splash window to overlap and conflict with the stays-on-top screen crop overlay window.

- **The Solution:** Modified the startup logic to detect if the splash window is still active during a hotkey trigger. If so, it instantly closes the splash window, avoiding coordinate and window-level overlap conflicts.

5. UI/UX Design & Aesthetic Specifications

SnapRoute implements modern web design principles inside a native Windows desktop environment:

- **Glassmorphism:** HUD containers feature translucent dark backgrounds (`rgba(12, 12, 20, 245)`) combined with neon-emerald neon borders (`rgba(0, 255, 136, 200)`).
- **Multi-Layered Glow Borders:** The selection crop rectangle features a three-layer paint style: a wide translucent outer glow (8px), a medium accent glow (4px), and a solid ultra-bright core (2px).
- **Opacity Animations:** Utilizing `QPropertyAnimation` on the widget's `windowOpacity` property, the HUD glides onto the screen with a smooth `180ms` fade-in, and closes with a clean `150ms` fade-out.